

Hojas:

Apellido y Nombre: Juan Antonio Pineda

1	2							TOTAL
4/10	8/10							10/10
13*	17							4 (Cuatro)

Parcial Assembler 05/JUN/2024

DESARROLLAR LOS SIGUIENTES EJERCICIOS EN ASSEMBLER DE LA MV IMPLEMENTADA EN EL CURSO, INCLUYENDO UNA ESPECIFICACIÓN DE INVOCACIÓN PARA CADA SUBROUTINA. RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE.

Ejercicio 1: Traducir el siguiente código en C a ASM de la máquina virtual.
AYUDA: los structs son referencias, no se traducen a assembler.

```
typedef struct _nnom{
    char inicial;
    char* nombre;
    struct _nnom* sig;
} nnom;

nnom* nnom_create(char* nombre){
    register nnom* r = alloc(9);
    if (r != NULL){
        r->inicial = *nombre;
        r->nombre = nombre;
        r->sig = NULL;
    }
    return r;
}

void nnom_add(nnom** header, nnom* nuevo){
    if (nuevo == NULL) return;
    register nnom** n = header;
    while (*n != NULL && (*n)->inicial < nuevo->inicial)
        n = &(*n)->sig;
    nuevo->sig = *n;
    *n = nuevo;
}
```

ALLOC: Desarrollar la subrutina alloc e indicar (en _start:) cómo debe estar inicializada la memoria.

Ejercicio 2: Utilizando las subrutinas del ejercicio 1, desarrollar la subrutina "GET_MAYORES" que permita, dada una fecha y un puntero a una lista dinámica enlazada donde cada nodo tiene un puntero a una estructura con datos de persona, crear otra lista (nnom*) que contenga los nombres y las iniciales de aquellas personas con 18 años o más a partir de la fecha pasada por parámetro. La función debe devolver el puntero a la nueva lista creada.

Persona:

	2 bytes	10 Bytes	21 Bytes	4 bytes
N	Id	DNI	Nombre	Nacimiento
	Entero	String	String	Tipo Fecha

Nodo lista personas:

Tipo Fecha:

	4 Bytes	4 Bytes
L	*Persona	*Siguiete
	Puntero a celda tipo N	Puntero a celda tipo L

	2 Bytes	1 Byte	1 Byte
F	Año	Mes	Día
	Entero	Entero	Entero

NOTA: Se exige un mínimo del 40% de desarrollo correcto de cada ejercicio para comenzar a tener puntaje del mismo.
 Realizar cada ejercicio en forma completa, precisa y con letra legible.

Juan Gonzalez 43771031

EBP

INT EQU 0 / nomb EQU 1 / sig EQU 2 / NULL EQU -1

1)

_START: MOV [EBP], ESP

ADD [EBP], 4

PUSH <CANT BYTES>

CALL ALLOC, DEVUELVE EN EAX EL PUNTERO

ADD SP, 4

PUSH <CANT BYTES>

CALL NOM-CREATE

ADD SP, 4

DEVUELVE EN EAX EL PUNTERO

EBP

MAX EQU 1024

ALLOC:

PUSH BP

MOV BP, SP

PUSH ECX

MOV EAX, NULL

MOV ECX, W[ESTB]

ADD ECX, [BP+8]

CMP ECX, MAX

JP FIN_ALLOC

MOV EAX, [EBP]

ADD W[ESTB], [BP+3]

FIN_ALLOC:

POP ECX

MOV SP, BP

POP BP

RET

NOM-CREATE: PUSH BP

MOV BP, SP

PUSH ECX

PUSH 9

CALL ALLOC

ADD SP, 4

CMP EAX, NULL

JZ FIN_CREATE

MOV ECX, [BP+3]

MOV [EAX+INT], B[ECX]

MOV [EAX+NOMB], ECX

MOV [EAX+SIG], NULL

FIN_CREATE: POP ECX

MOV SP, BP

POP BP

RET

PODÍA UTILIZAR
CX

LA INICIA QUEDA ASIGNADA
EN EL PUNTERO AL NOMBRE
QUE LUEGO SE PISA

```

push < *room >
push < * * room >
call room_add
add sp, 3

```

```

room_add:
    push bp
    mov bp, sp
    push eax
    push edx
    mov edx, [bp+12], *room
    cmp [bp+12], null
    jz fin_room
    mov ebx, [bp+12], **head

```

```

volver_while:
    cmp [ebx], null
    jz fin_while
    mov ecx, [ebx]
    cmp [ecx+init], [ecx+init]
    jnn fin_while

```

COMPARA LA INICIAL Y
PARTE DEL PUNTERO AL NOMBRE

```

    mov ebx, [ebx]
    add ebx, 516
    jmp volver_while
fin_while:
    mov [ecx+616], [ebx]
    mov [ebx], edx

```

```

fin_room:
    pop ecx
    pop edx
    pop ebx
    mov sp, bp
    pop bp

```